# Architecture Document


- Architecture Document
  - 1. Introduction
    - 1.1 Game Overview
    - 1.2 Scope of the Architecture document
  - 2. High-Level Architecture Overview
    - 2.1 Architectural Style
    - 2.2 Core Systems
  - 3. Core Modules and Classes
    - 3.1 Main Game
    - 3.2 Controller System
    - 3.3 Combat System
    - 3.4 Player System
    - 3.5 Enemy System
    - 3.6 Dungeon Generation
    - 3.7 UI System
    - 3.8 NPC Interaction System
  - 4. Class Relationship and Data Flow
    - 4.1 Class Diagram
  - 5. Sequences
    - 5.1 Player Movement
    - 5.2 Combat Sequence
    - 5.3 Dungeon Generation
  - 6. Future Expansions
  - 7. Appendix


## 1. Introduction

### 1.1 Game Overview

*Elemelons* is a 2D dungeon crawler that combines elemental spell-based combat with dynamic dungeon exploration. Players use the power all four elements - earth, fire, water and wind - to defeat monsters controlled by an evil witch. The main goal of the game is to retrieve the four stolen elemelons - the priced possessions of the king - and defeat the witch's army of monsters. The elemelons are found in elemental-themed, procedurally generated dungeons where the player has to navigate around traps, defeat creatures and solve small puzzles. The core gameplay is driven by elemental spell combat mechanics, and exploration, making each run through the dungeons a unique experience.

### 1.2 Scope of the Architecture document

This document presents the architectural design of the *Elemelons* game. It first gives a high-level view of the game's core systems and modules and shows the components necessary to establish the basic gameplay loop, including player controls, dungeon generation, combat interactions, NPC communication and basic user interface elements. This architecture provides a framework and document that allows for further expansion and detailization.

The document will outline each primary system and its respective modules and classes, describing how they interact to provide the game's gameplay loop. This architecture overview defines each system's purpose and scope within the game, as it serves as a guide for both current development as well as future enhancement to the game.

# 2. High-Level Architecture Overview

## 2.1 Architectural Style

In this game we use an Object-Oriented approach, encapsulating both state of data and methods within our class structure. The most important aspects of our object-oriented architecture include:

- Data and Behavior Composition: Classes encapsulate state and methods
- Inheritance: Game objects rely on inheritance hierarchies going from more abstract to concrete classes
- Abstract Base Classes: We want to use abstract base classes to create conceptual interfaces
- Modularity: Classes contain components of other classes for more reusability and generality

## 2.2 Core Systems

- Main Game Manager: switch scenes, progress levels, save game progress, start/continue game, change settings
- Controller System: user input system, keybinds, scene and player interaction
- Combat System: health and mana, damage calculations, spell combat
- Player System: Player controls, health and mana state, character movement, buffs and debuffs
- Enemy System: Movement behavior, health and mana state, combat behavior
- Dungeon Generation: Map and room creation, Map layout, Tile visualizaton, Player colliding
- UI System: health display, settings menu, mana display, spell charge display
- NPC Interaction System: NPC behavior, dialogue handling

# 3. Core Modules and Classes

## 3.1 Main Game

- Purpose: Manage scenes and game progress
- Key Classes:
  - **GameManager**: Responsible for managing the game's state and scene handling as well as game/level progression
  - **GameSaveProvider**: Saves game's progress and settings
- Interactions:
  - GameSaveProvider communicates the game's state and settings to the GameManager at game continuation
  - Items, NPC Interactions and Monster Combat Results can trigger GameManager's progression

## 3.2 Controller System

- Purpose: Manage the user input of keyboard and mouse
- Key Classes:
  - **InputManager**: Responsible for taking in mouse and keyboard input and mapping it to the respective systems

- Interactions:
    - GameSaveProvider saves the game settings and keybinds which are mapped to their correct functionality
    - The InputManager maps keystrokes to the PlayerMovement class of the player
    - The InputManager maps keystrokes and mouse movement to the SpellCasting class of the player
    - Some special keys communicate to the UI System to open the settings menu or start/continue game with the GameManager

### 3.3 Combat System

- Purpose: Handle damage calculations and health/mana management of characters, spell casting
- Key Classes:
    - **CombatManager**: Calculate damage based on the combatting characters and manage health and mana of characters
    - **Health**: Handle health value, subtract or add to it, regenerate over time
    - **Mana**: Handle mana value, subtract or add to it, regenerate over time
    - **SpellCasting**: handle collisions, animations, starting of cast spells
- Interactions:
    - CombatManager uses data of the Player character and Enemy characters to calculate damage
    - CombatManager calculates used Mana and subtracts Health if damage is dealt in combat
    - Player and Enemy classes have a Health and Mana object
    - The Player and some Enemy classes have different Spells which they can utilize in combat

### 3.4 Player System

- Purpose: Handle the players controls and movement, manage the player state of health/mana and debuffs
- Key Classes:
    - **Player**: Handles player actions and store the player state as well as spell charges for casting
    - **PlayerMovement**: Move the character on the board and handle collisions with the environment
- Interactions:
    - PlayerMovement uses inputs from the InputManager
    - Player contains Health and Mana Info
    - Player interacts with GameManager for game state updates
    - Player uses spell charges to utilize different spells with the SpellCasting class

### 3.5 Enemy System

- Purpose: Handle the monsters movements and combat as well as health state
- Key Classes:
    - **Enemy**: Base class for monster enemies, with subclasses for specific monsters
- Interactions:
    - Enemy contains Health and Mana info
    - Enemies will fight the player and use the CombatManager to execute combat
    - Some enemies can utilize spells using the SpellCasting class

### 3.6 Dungeon Generation

- Purpose: Create procedural dungeons with both randomization and handcrafted design elements
- Key Classes:
  - **DungeonGenerator**: Base class for dungeon generation, uses different map generators and tilemap visualization
  - **MapGenerator**: Base class of a map generator, creates rooms, floors, walls and decoration as well as monster spawn spots
  - **TilemapVisualizer**: Base class of a tilemap visualizer, places tiles for floor and walls as well as decoration
  - **MapData**: Data class to store rooms, floor and wall positions, decoration, door and monster spawn positions
  - **TilemapManagement**: Save tilemaps in data, open up tilemaps from data
- Interactions:
  - MapGenerator provides MapData to TilemapVisualizer, which renders it on screen
  - MapData is used by the MainGame manager to spawn characters in the scene
  - The Player collides and moves through the tilemaps created by the TilemapVisualizer

### 3.7 UI System

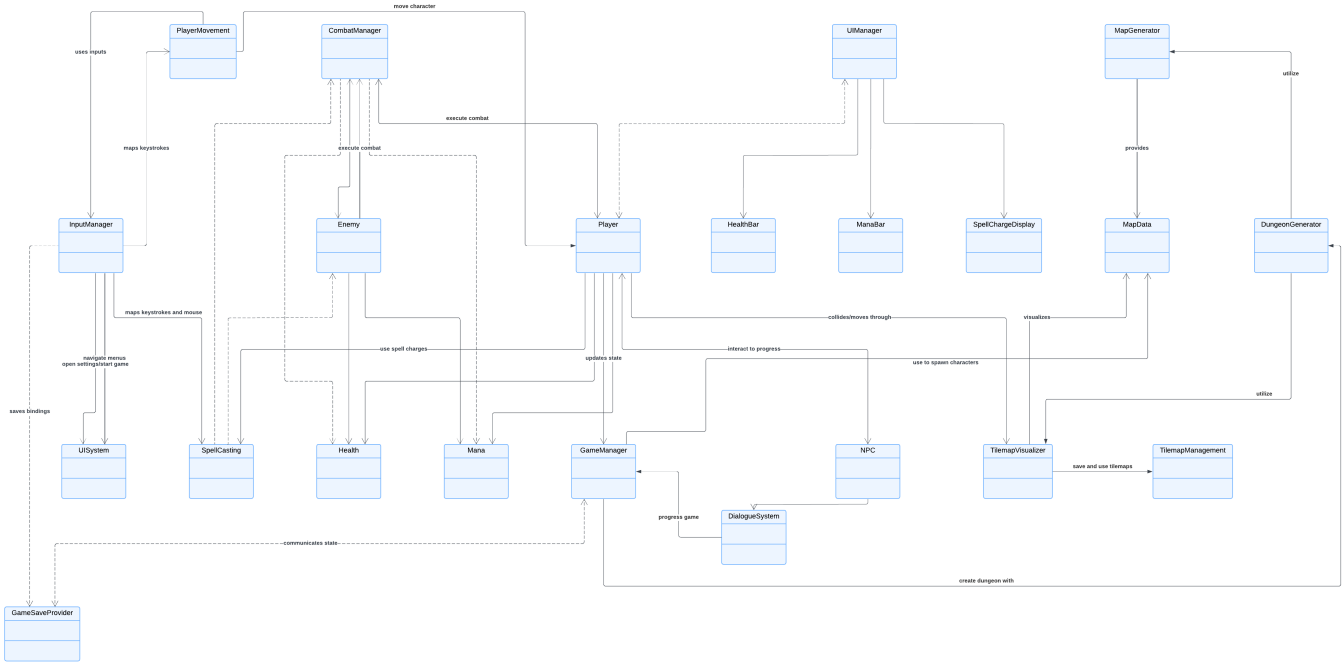
- Purpose: Display health, mana and spell charges of the player, show menu ui elements
- Key Classes:
  - **UIManager**: Manages the HUD, main menus and settings screens
  - **HealthBar**: Displays the health points of the player
  - **ManaBar**: Displays the mana stars of the player
  - **SpellChargeDisplay**: Shows the spell charges displayed above the players head
- Interactions:
  - UIManager updates the health and mana elements based on changes of Player Health/Mana
  - Main Menus and settings can be navigated using the InputManager
  - The InputManager can manage the players spell charges which are visualized using the UIManager

### 3.8 NPC Interaction System

- Purpose: Handles NPC behavior and dialogues
- Key Classes:
  - **King**: Only NPC character so far, player talks to the king to enter dungeons and progress through the game
  - **DialogueSystem**: Stores dialogue options with the King and other important boss enemies which the player can access
- Interactions:
  - Player can interact with the King to progress the game state using the GameManager
  - Interactions with the King or some Bosses will give a dialogue using the DialogueSystem

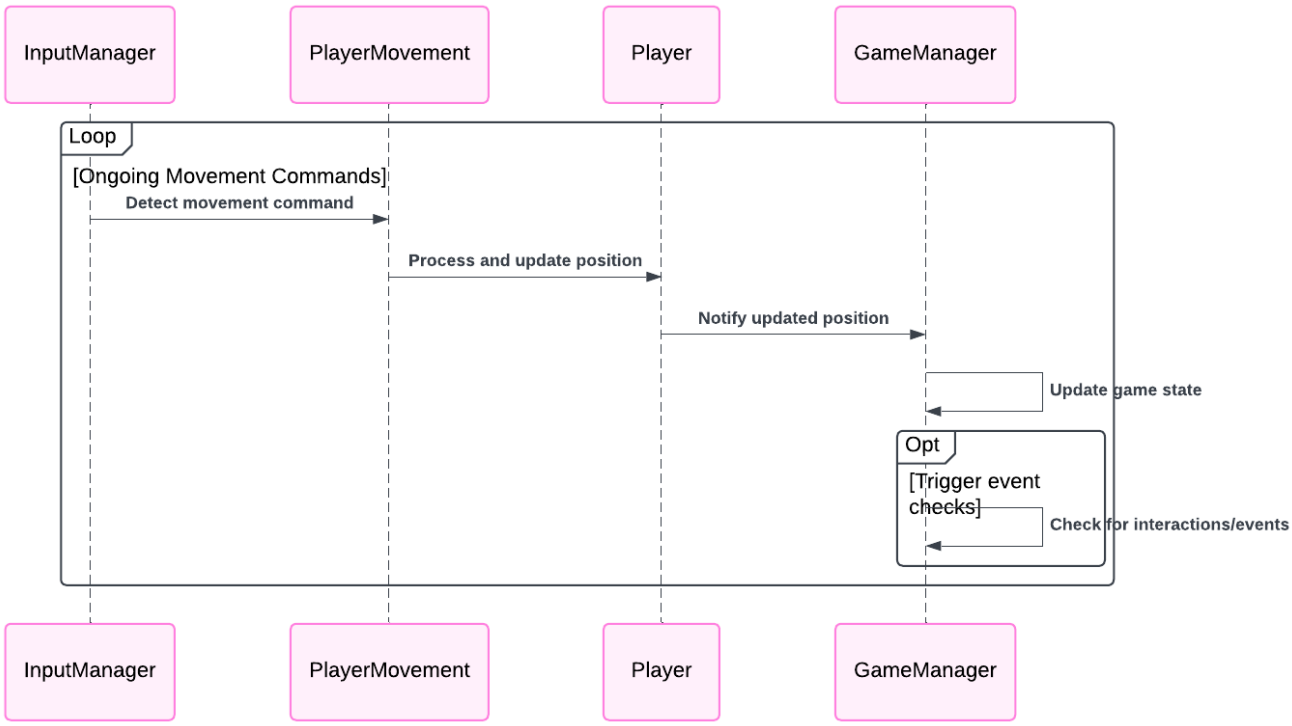
## 4. Class Relationship and Data Flow

### 4.1 Class Diagram

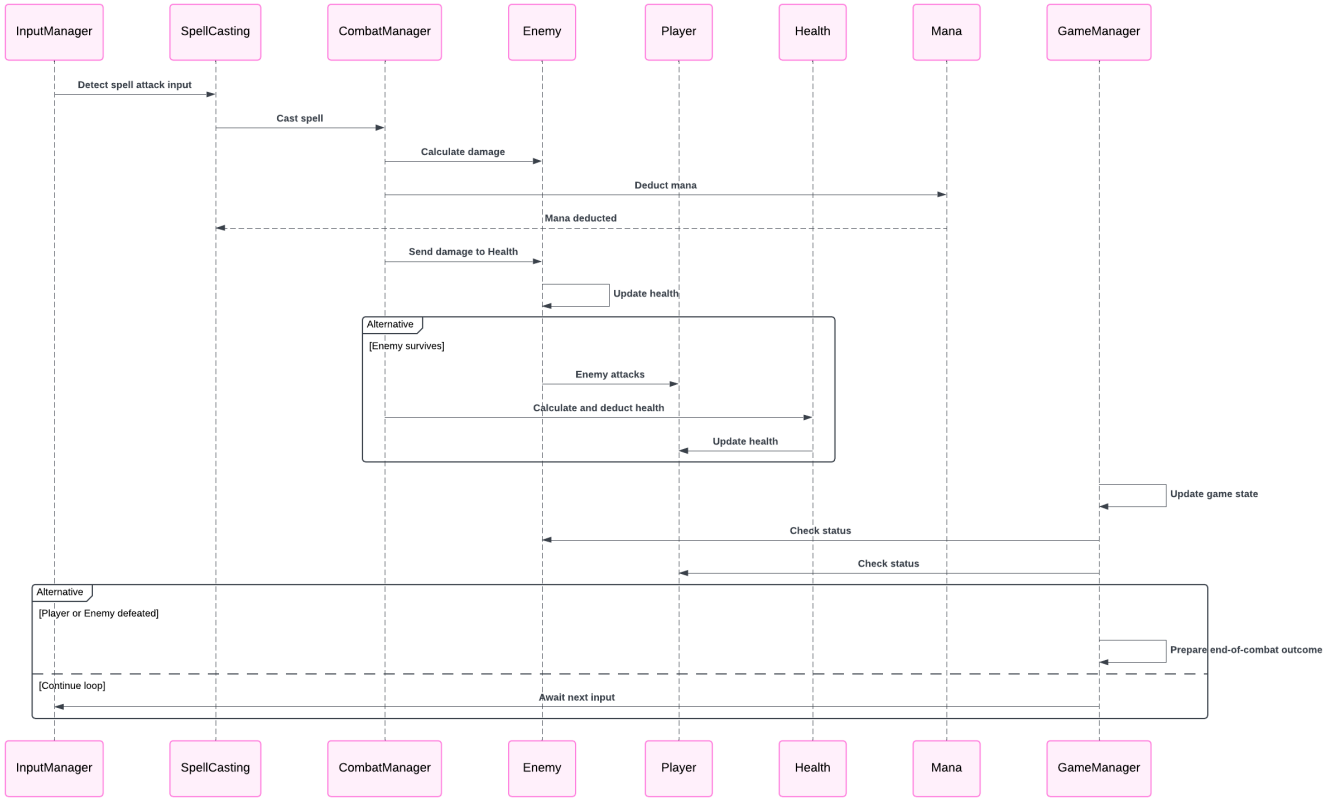



# 5. Sequences

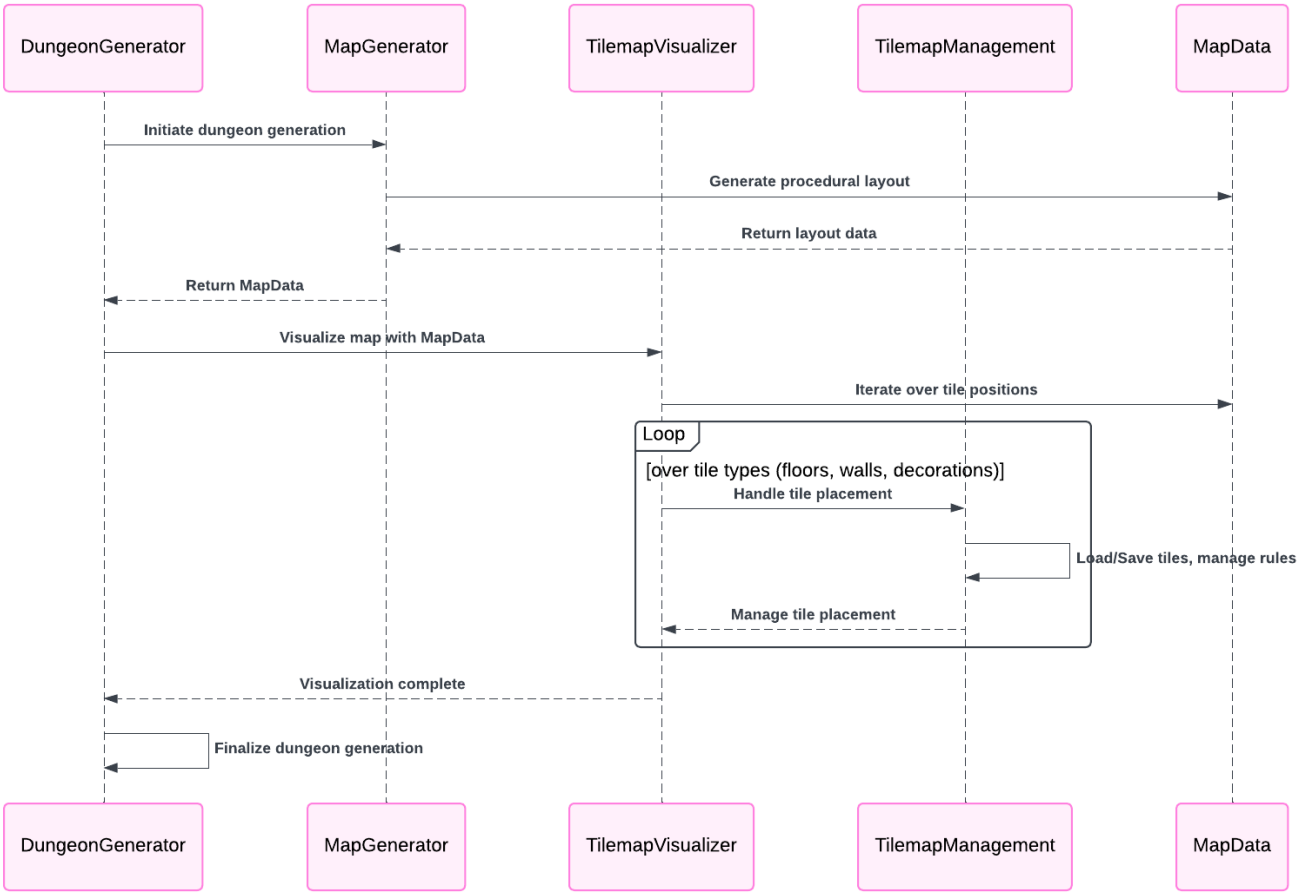
## 5.1 Player Movement




## 5.2 Combat Sequence




## 5.3 Dungeon Generation




# 6. Future Expansions

- 5.4 Dialogue Interaction Sequence and Dialog Requirements

# 7. Appendix